



C++ PROGRAMMING (335)

STATE – 2019

Production Portion:

Job 1: Department Graph _____ (320 points)

TOTAL POINTS _____ (*320 points*)

**Graders: Please double check and verify all scores
and answer keys!**

Property of Business Professionals of America.
May be reproduced only for use in the Business Professionals of America
Workplace Skills Assessment Program competition.



Your application will be graded on the following criteria:

Solution and Project

Project was found on the flash drive _____ 10 points
Project is named following the naming convention _____ 10 points

Program Execution

Program runs _____ 20 points
If program does not execute, then remaining items in this section are not scored.

The program gracefully handles an empty or missing input file _____ 10 points
The program allows for both types of word collection _____ 20 points
The program displays a puzzle in all upper case letters with the words placed in it _____ 20 points
The program displays the same puzzle with the words in lower case _____ 20 points
The program displays the list of words found in the puzzle _____ 10 points
The program displays a prompt for collecting words from the user that includes how many words left to enter. _____ 10 points

Source Code Review

Contestant ID is commented at the top of the program _____ 5 points
Code is commented at the top, for each method and as needed _____ 15 points
Directional code is commented ie. Left to Right _____ 24 points
Code uses reasonable and consistent variable naming conventions _____ 15 points
The program reads the words into the correct data structure _____ 10 points
The program gracefully handles remaining error checking _____ 20 points
The program generates random capital letters in the correct data structure _____ 20 points
The program deals with the difference between random letters and letters from a word _____ 20 points
A method called "Build_Clean_Puzzle" is implemented _____ 10 points
A method called "Read_Words_From_File" is implemented _____ 20 points
A method called "User_Words_Input" is implemented _____ 30 points
A method called "Hide_Words_In_Puzzle" is implemented _____ 80 points
A method called "Display_Puzzle_Hidden" is implemented _____ 20 points
A method called "Display_Puzzle_Show" is implemented _____ 20 points
The program uses the bool data type in a logical manor _____ 10 points

Total _____/449 points



C++ PROGRAMMING - STATE 2019

ANSWER KEY

Page 3 of 10

```
// Word Puzzle.cpp : Defines the entry point for the console application.
//

// Program:Word Puzzle
// By: Bryan J. Baker - BakerLand Labs

#include "stdafx.h"
#include <iostream>
#include <iomanip>
#include <cstdlib>
#include <ctime> // needed for rand()% seed
#include <string>
#include <vector>
#include <fstream>
#include <conio.h>

using namespace std;
//Const Section

//Var Section
int Choice;
string Trash, Title;
double R;
// Vector to hold the words
vector <string> Words(20);

// Arrays to hold the puzzle chars
char Puzzle_Grid[20][20];
// Array to hold flag of used or not use location
bool Bool_Grid[20][20];

// This method build a clean puzzle to hide the words in
void Build_Clean_Puzzle(char Puzzle_Grid[][20], bool Bool_Grid[][20])
{
    // Local Vars
    int X, Y;

    // Go through the arrays one spot at a time
    for (X = 0; X<20; X++)
        for (Y = 0; Y<20; Y++)
        {
            // Sets a random capital letter in the char array.
            Puzzle_Grid[X][Y] = char((rand() % 26) + 65);
            // Sets the bool array to all false
            Bool_Grid[X][Y] = false;
        }
}
```



C++ PROGRAMMING - STATE 2019

ANSWER KEY

Page 4 of 10

```
// Reads the words in from the text file.
void Read_Words_From_File(vector<string> & Words, string & Title)
{ // Local Vars
    int I, Count;
    // input file stream
    ifstream In_File;
    vector<string> Temp_List;

    // open the file
    In_File.open("SocStudy_Vocab.txt");
    // Collect the number of words in the file.
    In_File >> Count;
    // To read only the vocab words
    Count--;
    // Resize the Temp_List vector to hold all the words in the file.
    Temp_List.resize(Count);
    Words.resize(Count);

    // if the file there and not empty
    if (!In_File.fail())
    {
        // Read the words into Temp_List
        for (I = 0; I < Count; I++)
            In_File >> Temp_List[I];
        // To read the puzzle title
        In_File >> Title;
        // Reset Count
        Count = 0;
        // Check the list for words to large or to small
        for (I = 0; I < Temp_List.size(); I++)
        {
            if (Temp_List[I].length() > 2 && Temp_List[I].length() < 9)
            {
                Words[Count] = Temp_List[I];
                Count++;
            }
        }
        // Resize Words to the size of the words in it.
        Words.resize(Count);
    }
    else // File is missing so stop the program
    {
        cout << "File is missing. Press enter to close program." << endl;
        getchar();
        exit(1);
    }
    // Close the file stream
    In_File.close();
}
```



C++ PROGRAMMING - STATE 2019

ANSWER KEY

Page 5 of 10

```
// Collect the words from the user
void User_Words_Input(vector<string> & Words, string &Title)
{ // Local Vars
    int I, Num_Words, W;

    // Collect the number of words the user wants to enter.
    // Check they enter a valid answer.
    do {
        cout << endl << endl;
        cout << "Please enter the number of words in the puzzle";
        cout << " (Min 0 & Max 20):";
        cin >> Num_Words;
        if (Num_Words < 0 || Num_Words > 20)
            cout << "Please enter only numbers from 0 to 20" << endl;
    } while (Num_Words < 0 || Num_Words > 20);
    // Used to clear the input buffer for future user input.
    getchar();

    // Check if they entered a zero(0) to make an empty puzzle.
    if (Num_Words > 0)
    {
        // Change the vector size to the number of words the user wants.
        Words.resize(Num_Words);

        // Collect the Puzzle's title. Checks for empty strings.
        do {
            cout << endl << "Please enter the puzzle title:";
            getline(cin, Title);
        } while (Title == "");

        // Collect the words and error check for allowed word lengths.
        for (I = 0; I < Num_Words; I++)
        {
            // Error check the word length
            do {
                cout << "Please enter a word of size from 3 to 8";
                cout << " letters long in all lower case letters." << endl;
                cout << "#" << I + 1 << ": ";
                getline(cin, Words[I]);
            } while (Words[I].length() < 2 || Words[I].length() > 8);

            // Force the word into lower case letters
            for (W = 0; W < Words[I].length(); W++)
                Words[I][W] = tolower(Words[I][W]);
        }
    }
}
```



C++ PROGRAMMING - STATE 2019

ANSWER KEY

Page 6 of 10

// Method to hide the words

```
void Hide_Words_In_Puzzle(vector <string> Words, char  Puzzle_Grid[][20], bool Bool_Grid[][20])
```

```
{// Local Vars
```

```
    int I, J, X, Y;
```

```
    bool Again;
```

```
    // Places the words in the puzzle.
```

```
    // Randomly places word in any of the 8 directions
```

```
    for (I = 0; I<Words.size(); I++)
```

```
        switch (rand() % 8)
```

```
        {
```

```
        case 0: //Down Direction
```

```
            // Looks for a clean spot in the puzzle to place the word.
```

```
            // Loops till one is found.
```

```
            // Check ALL the spots are good BEFORE placing the word.
```

```
            do {
```

```
                X = rand() % (20 - Words[I].length() - 1);
```

```
                Y = rand() % (20);
```

```
                Again = false;
```

```
                for (J = 0; J<Words[I].length(); J++)
```

```
                    if (Bool_Grid[X + J][Y] == true && Puzzle_Grid[X + J][Y] !=
```

```
Words[I][J])
```

```
                        Again = true;
```

```
            } while (Again);
```

```
            // Places the word in the puzzle and places the corresponding spot in the bool array
```

```
to true
```

```
            // for a used spot.
```

```
            for (J = 0; J<Words[I].length(); J++)
```

```
            {
```

```
                Puzzle_Grid[X + J][Y] = Words[I][J];
```

```
                Bool_Grid[X + J][Y] = true;
```

```
            }
```

```
            break;
```

```
        case 1: //Right Direction
```

```
        do {
```

```
            X = rand() % (20);
```

```
            Y = rand() % (20 - Words[I].length() - 1);
```

```
            Again = false;
```

```
            for (J = 0; J<Words[I].length(); J++)
```

```
                if (Bool_Grid[X][Y + J] == true && Puzzle_Grid[X][Y + J] !=
```

```
Words[I][J])
```

```
                        Again = true;
```

```
            } while (Again);
```

```
            for (J = 0; J<Words[I].length(); J++)
```

```
            {
```

```
                Puzzle_Grid[X][Y + J] = Words[I][J];
```



```

        Bool_Grid[X][Y + J] = true;
    }
    break;
case 2: //Up Direction
    do {
        X = rand() % (20 - Words[I].length() - 1) + Words[I].length() - 1;
        Y = rand() % (20);
        Again = false;
        for (J = Words[I].length() - 1; J >= 0; J--)
            if (Bool_Grid[X - J][Y] == true && Puzzle_Grid[X - J][Y] !=
Words[I][J])
                Again = true;
        } while (Again);

    for (J = Words[I].length() - 1; J >= 0; J--)
    {
        Puzzle_Grid[X - J][Y] = Words[I][J];
        Bool_Grid[X - J][Y] = true;
    }
    break;
case 3: //Left Direction
    do {
        X = rand() % (20);
        Y = rand() % (20 - Words[I].length() - 1) + Words[I].length() - 1;
        Again = false;
        for (J = Words[I].length() - 1; J >= 0; J--)
            if (Bool_Grid[X][Y - J] == true && Puzzle_Grid[X][Y - J] !=
Words[I][J])
                Again = true;
        } while (Again);

    for (J = Words[I].length() - 1; J >= 0; J--)
    {
        Puzzle_Grid[X][Y - J] = Words[I][J];
        Bool_Grid[X][Y - J] = true;
    }
    break;
case 4: //Up and Right Direction
    do {
        X = rand() % (20 - Words[I].length() - 1) + Words[I].length() - 1;
        Y = rand() % (20 - Words[I].length() - 1);
        Again = false;
        for (J = 0; J < Words[I].length(); J++)
            if (Bool_Grid[X - J][Y + J] == true &&
                Puzzle_Grid[X - J][Y + J] != Words[I][J])
                Again = true;
        } while (Again);
    for (J = 0; J < Words[I].length(); J++)

```



```

        {
            Puzzle_Grid[X - J][Y + J] = Words[I][J];
            Bool_Grid[X - J][Y + J] = true;
        }
        break;
case 5: //Down and Right Direction
do {
    X = rand() % (20 - Words[I].length() - 1);
    Y = rand() % (20 - Words[I].length() - 1);
    Again = false;
    for (J = 0; J < Words[I].length(); J++)
        if (Bool_Grid[X + J][Y + J] == true &&
            Puzzle_Grid[X + J][Y + J] != Words[I][J])
            Again = true;
    } while (Again);
    for (J = 0; J < Words[I].length(); J++)
    {
        Puzzle_Grid[X + J][Y + J] = Words[I][J];
        Bool_Grid[X + J][Y + J] = true;
    }
    break;
case 6: //Down and Left Direction
do {
    X = rand() % (20 - Words[I].length() - 1);
    Y = rand() % (20 - Words[I].length() - 1) + Words[I].length() - 1;
    Again = false;
    for (J = 0; J < Words[I].length(); J++)
        if (Bool_Grid[X + J][Y - J] == true &&
            Puzzle_Grid[X + J][Y - J] != Words[I][J])
            Again = true;
    } while (Again);
    for (J = 0; J < Words[I].length(); J++)
    {
        Puzzle_Grid[X + J][Y - J] = Words[I][J];
        Bool_Grid[X + J][Y - J] = true;
    }
    break;
case 7: //Up and Left Direction
do {
    X = rand() % (20 - Words[I].length() - 1) + Words[I].length() - 1;
    Y = rand() % (20 - Words[I].length() - 1) + Words[I].length() - 1;
    Again = false;
    for (J = Words[I].length() - 1; J >= 0; J--)
        if (Bool_Grid[X - J][Y - J] == true && Puzzle_Grid[X - J][Y - J] !=
Words[I][J])
            Again = true;
    } while (Again);
    for (J = Words[I].length() - 1; J >= 0; J--)
    {

```



C++ PROGRAMMING - STATE 2019

ANSWER KEY

Page 9 of 10

```
Puzzle_Grid[X - J][Y - J] = Words[I][J];
Bool_Grid[X - J][Y - J] = true;
    }
    break;
}

// Display the Puzzle with the words hidden
void Display_Puzzle_Hidden(vector<string> Words, char Puzzle_Grid[][20])
{
    // Local Vars
    int X, Y;
    // Display the Puzzle Title
    cout << setw(20) << Title << setw(44) << "Words in Puzzle" << endl;

    // Display the Puzzle in ALL CAPS
    for (X = 0; X < 20; X++)
    {
        for (Y = 0; Y < 20; Y++)
        {
            // convert ALL chars to CAPS on screen
            cout << char(toupper(Puzzle_Grid[X][Y])) << " ";

            // Display the puzzle's words to the right
            if (X < Words.size())
                cout << setw(5) << X + 1 << "." << Words[X] << endl;
            else
                cout << endl;
        }
        // Hold the screen till the user is ready to continue.
        cout << "Please press enter to see the hidden words.";
        getchar();
    }

    // Display the Puzzle with the words in lower case letters to be seen.
    void Display_Puzzle_Show(vector<string> Words, char Puzzle_Grid[][20])
    {
        // Local Vars
        int X, Y;
        // Draw 40 empty lines so the user sees a second puzzle
        for (X = 0; X < 40; X++)
            cout << endl;
        // Again display the puzzle title
        cout << setw(20) << Title << setw(44) << "Words in Puzzle" << endl;
        // Display the Puzzle and the words found in it
        for (X = 0; X < 20; X++)
        {
            for (Y = 0; Y < 20; Y++)
                cout << Puzzle_Grid[X][Y] << " ";
            // Display the words to the right of the puzzle
            if (X < Words.size())
                cout << setw(5) << X + 1 << "." << Words[X] << endl;
            else
                cout << endl;
        }
    }
}
```



C++ PROGRAMMING - STATE 2019

ANSWER KEY

Page 10 of 10

```
        cout << endl;
    }

    // Wait till the user is ready to exit the program.
    cout << "Please press enter to exit.";
    getchar();
}

void main()
{
    // Default Title
    Title = "Chapter#0";
    // Sets the random seed
    srand((unsigned)(time(0)));

    // Collect where the words come from.
    do {
        cout << "1. User word input" << endl;
        cout << "2. File word input" << endl;
        cin >> Choice;
        if (Choice < 0 || Choice > 2)
            cout << "Please enter only the numbers 1 or 2" << endl << endl;

    } while (Choice < 0 || Choice > 2);
    getchar(); // Collect extra return - Trash

    // Call the needed method
    if (Choice == 1)
        User_Words_Input(Words, Title);
    else
        Read_Words_From_File(Words, Title);

    // Call each method.
    Build_Clean_Puzzle(Puzzle_Grid, Bool_Grid);
    Hide_Words_In_Puzzle(Words, Puzzle_Grid, Bool_Grid);
    Display_Puzzle_Hidden(Words, Puzzle_Grid);
    Display_Puzzle_Show(Words, Puzzle_Grid);

} // end main
```